

Lab 6

Jooho Oh

02-22-2023

**** NOTE: **** Please undo `eval = FALSE` in the following chunk:

```
options(digits = 3)
knitr::opts_chunk$set(echo = TRUE)
```

Submission Instructions

This lab (and all labs) will be graded for correctness. To submit the lab:

1. Once all code is completed and saved, in the RStudio ribbon, click Knit > Knit to PDF.
2. If you have trouble knitting **anything** to PDF, please work with the Instructor or GSI to resolve the issue. It is okay to instead knit to HTML, then convert to PDF using Adobe or other software.
3. If there is any handwritten work for the lab, combine the handwritten work with the knitted PDF into a single PDF file.
4. Submit the PDF to Gradescope, **selecting** pages for questions. *You will lose half a point on the lab if you don't select pages.*

Lab 6 is due to Gradescope by Sunday, February 26th at 11:59PM PT.

Today's lab we will return to real data to:

- show how to regress on categorical variables
- implement the incremental F-test

Part I: Categorical Regression

Today we will be looking at a dataset of trait information (on a likert scale) for dog breeds from the American Kennel Club (AKC), along with the breed popularity ranks. This data comes from the TidyTuesday package and is accessible directly here.

Here are the data sets we will be inspecting today:

```
breed_traits <- read.csv("~/Desktop/spring 2023/stat 151a/lab6/breed_traits.csv")
trait_description <- read.csv("~/Desktop/spring 2023/stat 151a/lab6/trait_description.csv")
breed_rank_all <- read.csv("~/Desktop/spring 2023/stat 151a/lab6/breed_rank_all.csv")
```

The data is slightly messy as we have read it.

Q1: Each data set has a first column called X. What does this correspond to?

ANSWER: The csv's have row numbers in the first column that is read in.

You will also note that R does not allow variable names to begin with a number. In `breed_rank_all`, R has by default put an X in front of each such variable.

Q2: Clean the rank variable names in `breed_rank_all`, such that `X2013.Rank` becomes `rank_2013`, for example.

Hint: `names()` will allow you to manipulate column names. `gsub()` and `grepl()` are helpful regular expression tools, though not necessary to do this exercise. `paste()` allows you to concatenate strings.

```
## First we will create new DFs in case we make a mistake, we won't have to reload the data.
breed_traits_clean <- breed_traits %>% dplyr::select(-X)
trait_description_clean <- trait_description %>% dplyr::select(-X)
breed_rank_all_clean <- breed_rank_all %>% dplyr::select(-X)
```

YOUR CODE HERE:

```
new_names = paste("rank", seq(2013,2020), sep="_")
names(breed_rank_all_clean)[2:9] = new_names
```

```
#names(breed_rank_all_clean) <-
# gsub("X([[:digit:]]+)\.Rank", "rank_\1",
#      names(breed_rank_all_clean))
```

```
head(breed_rank_all_clean)
```

```
##           Breed rank_2013 rank_2014 rank_2015 rank_2016 rank_2017
## 1 Retrievers (Labrador)      1        1        1        1        1
## 2      French Bulldogs     11        9        6        6        4
## 3 German Shepherd Dogs      2        2        2        2        2
## 4 Retrievers (Golden)       3        3        3        3        3
## 5      Bulldogs            5        4        4        4        5
## 6      Poodles             8        7        8        7        7
```

```
## rank_2018 rank_2019 rank_2020
```

```
## 1      1      1      1
## 2      4      4      2
## 3      2      2      3
## 4      3      3      4
## 5      5      5      5
## 6      7      6      6
```

```
##                                     links
```

```
## 1 https://www.akc.org/dog-breeds/labrador-retriever/
## 2 https://www.akc.org/dog-breeds/french-bulldog/
## 3 https://www.akc.org/dog-breeds/german-shepherd-dog/
## 4 https://www.akc.org/dog-breeds/golden-retriever/
## 5 https://www.akc.org/dog-breeds/bulldog/
## 6 https://www.akc.org/dog-breeds/poodle-standard/
```

```
##                                     Image
```

```
## 1 https://www.akc.org/wp-content/uploads/2017/11/Labrador-Retriever-illustration.jpg
## 2 https://www.akc.org/wp-content/uploads/2017/11/French-Bulldog-Illo-2.jpg
```

```
## 3      https://www.akc.org/wp-content/uploads/2017/11/German-Shepherd-Dog-Illo-2.jpg
## 4      https://www.akc.org/wp-content/uploads/2017/11/Golden-Retriever-Illo-2.jpg
## 5      https://www.akc.org/wp-content/uploads/2017/11/Bulldog-Illo-2.jpg
## 6      https://www.akc.org/wp-content/uploads/2017/11/Standard-Poodle-illustration.jpg
```

Finally, we will use the janitor package to clean up the other names:

```
breed_rank_all_clean <-
  breed_rank_all_clean %>%
  janitor::clean_names()

breed_traits_clean <-
  breed_traits_clean %>%
  janitor::clean_names()

trait_description_clean <-
  trait_description_clean %>%
  janitor::clean_names()
```

breed_traits_clean is of course interesting background info that you should feel free to check out. For today's analysis we will focus on the 2020 ranks.

Q3: Create new dataset, breed_data, that has the 2020 ranks as the first column, then all the columns in breed_traits_clean.

Hint: The data set rows are in the same order, so you can just cbind().

```
## YOUR CODE HERE:
breed_data <- cbind(dplyr::select(breed_rank_all_clean,rank_2020),breed_traits_clean)
head(breed_data)
```

```
##   rank_2020      breed affectionate_with_family
## 1         1 Retrievers (Labrador)              5
## 2         2      French Bulldogs              5
## 3         3 German Shepherd Dogs              5
## 4         4 Retrievers (Golden)              5
## 5         5      Bulldogs                  4
## 6         6      Poodles                    5
##   good_with_young_children good_with_other_dogs shedding_level
## 1                        5                      5              4
## 2                        5                      4              3
## 3                        5                      3              4
## 4                        5                      5              4
## 5                        3                      3              3
## 6                        5                      3              1
##   coat_grooming_frequency drooling_level coat_type coat_length
## 1                        2                2 Double      Short
## 2                        1                3 Smooth      Short
## 3                        2                2 Double Medium
## 4                        2                2 Double Medium
## 5                        3                3 Smooth      Short
## 6                        4                1  Curly      Long
```

```
##   openness_to_strangers playfulness_level watchdog_protective_nature
## 1           5           5           3
## 2           5           5           3
## 3           3           4           5
## 4           5           4           3
## 5           4           4           3
## 6           5           5           5
##   adaptability_level trainability_level energy_level barking_level
## 1           5           5           5           3
## 2           5           4           3           1
## 3           5           5           5           3
## 4           5           5           3           1
## 5           3           4           3           2
## 6           4           5           4           4
##   mental_stimulation_needs
## 1           4
## 2           3
## 3           5
## 4           4
## 5           3
## 6           5
```

```
## The following breed doesn't seem to have data
```

```
breed_data <- breed_data %>%
  dplyr::filter(rank_2020 != 167)
```

Most of the trait variables are ordinal, on a scale from 1 to 5. 5 tends to designate a higher level of the trait, for example more shedding or more affection.

These trait variables are usually not best treated as continuous variables. In particular we might be interested if there is a larger jump in dog rank from level 3 to 4 than from level 2 to 3, for instance.

Q4: Briefly explain why leaving the trait variables as continuous won't help us explore the previous statement.

YOUR ANSWER HERE: By keeping it continuous, we won't be able to model any non-linearity between our data.

Please review the following code that converts each ordinal variable to a character:

```
## This function loops through the columns of the data set:
```

```
is_ordinal <- sapply(breed_data, function(x) {
  print(max(x))
  if(is.numeric(x)) {
    return(max(x) == 5)
  } else{
    return(FALSE)
  }
})
```

```
## [1] 195
## [1] "Yorkshire Terriers"
## [1] 5
```

```
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] "Wiry"
## [1] "Short"
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
## [1] 5
```

```
ord_vars <- names(breed_data)[is_ordinal]
```

```
## This function lets us apply the same mutation to several variables at once.
breed_data <- breed_data %>%
  dplyr::mutate_at(ord_vars, as.character)
```

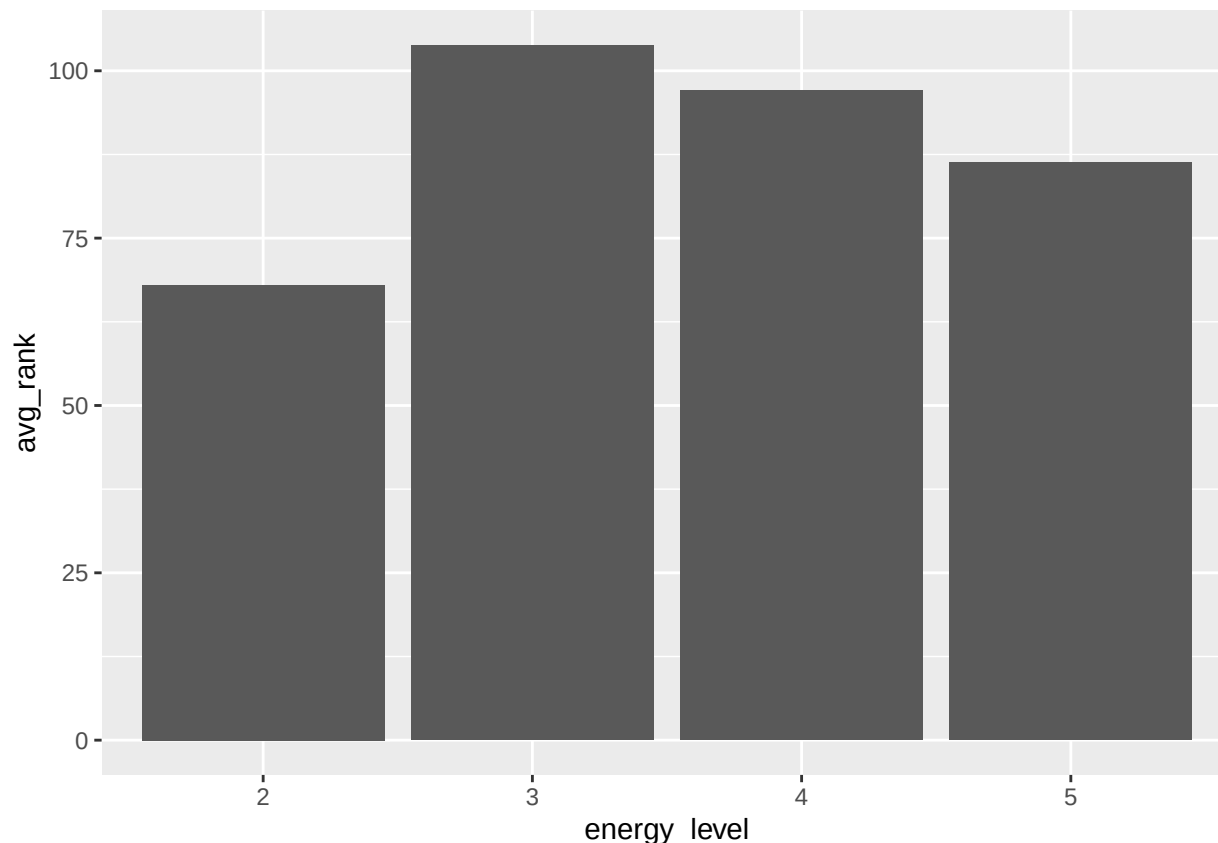
By converting the ordinal variables to strings, we are telling R to treat the values as separate levels rather than as a continuous variable.

Q5: Create a figure showing the average rank for each level of a trait of your interest. What do you see? Does there appear to be nonlinearity between the levels?

Hint: `group_by()` and `summarise()` are your friend.

```
## YOUR CODE HERE:
```

```
new_df = dplyr::summarise(dplyr::group_by(breed_data, energy_level), avg_rank = mean(rank_2020))
ggplot(data = new_df, aes(x=energy_level, y = avg_rank)) + geom_bar(stat = "identity")
```



YOUR ANSWER HERE: There is non-linearity between our categorical variable and average rank.

When we are modeling a categorical variable, we will create a *dummy variable* (or binary variable) for each level of the category. These will be a set of covariates that are jointly added to the model. For example, for `good_with_other_dogs`, a model will create variables:

$$good_with_other_dogs_level_i = \begin{cases} 1, & good_with_other_dogs = i \\ 0, & good_with_other_dogs \neq i \end{cases}$$

When we feed `lm()` a character variable, it will automatically create dummy variables for each level of the character variable.

Q6: Run an `lm()` of `rank_2020` on `good_with_other_dogs` and summarize.

YOUR CODE HERE:

```
breed_model_other_dogs <- lm(rank_2020 ~ good_with_other_dogs, data = breed_data)
summary(breed_model_other_dogs)
```

```
##
## Call:
## lm(formula = rank_2020 ~ good_with_other_dogs, data = breed_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
```

```
## -97.17 -48.83 -1.58 47.60 105.73
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      77.33      32.65   2.37   0.019 *
## good_with_other_dogs2  37.67      41.30   0.91   0.363
## good_with_other_dogs3  22.75      33.06   0.69   0.492
## good_with_other_dogs4   4.94      34.81   0.14   0.887
## good_with_other_dogs5  20.84      33.70   0.62   0.537
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 56.6 on 189 degrees of freedom
## Multiple R-squared:  0.0141, Adjusted R-squared: -0.00674
## F-statistic: 0.677 on 4 and 189 DF, p-value: 0.609
```

Note the average rank of level 1 for good with other dogs:

```
breed_data %>%
  dplyr::filter(good_with_other_dogs == "1") %>% summarise(mean(rank_2020))

##   mean(rank_2020)
## 1              77.3
```

Q7: Why is there no variable for `good_with_other_dogs1` in the `lm()` summary?

YOUR ANSWER HERE: `good_with_other_dogs1` introduces a dependent explanatory variable. However, the average rank of `good_with_other_dogs1` factored into the intercept.

Note that if we included a dummy variable for **every** level in a categorical variable, then there would be linear dependence. I.e., if we know that a breed does not belong to levels 2, 3, 4, or 5, then it **must** belong to group 1.

Q8: Use the model summary to provide the average rank of `good_with_other_dogs2`

YOUR ANSWER HERE: $77.33 + 37.67 = 115$

We will discuss categorical modeling in more detail in the coming week. Let's conclude today's analysis by mechanically running the incremental F-test.

Let's say we are interested in seeing if including `good_with_young_children` **along with** `good_with_other_dogs` in the model will provide us significantly more information than just using `good_with_other_dogs` in order to explain breed popularity.

Below, we run a model on both of these variables:

```
breed_model_both_vars <- lm(rank_2020 ~ good_with_other_dogs + good_with_young_children, data = breed_data)
summary(breed_model_both_vars)

##
## Call:
## lm(formula = rank_2020 ~ good_with_other_dogs + good_with_young_children,
##     data = breed_data)
```

```
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -104.63  -46.17   -3.59   47.14  119.65
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)         11.35      63.87   0.18   0.86
## good_with_other_dogs2  26.03      41.17   0.63   0.53
## good_with_other_dogs3  22.65      32.25   0.70   0.48
## good_with_other_dogs4   9.31      34.03   0.27   0.78
## good_with_other_dogs5  32.58      33.05   0.99   0.33
## good_with_young_children3 75.63      55.45   1.36   0.17
## good_with_young_children4 80.62      58.98   1.37   0.17
## good_with_young_children5 46.70      55.69   0.84   0.40
##
## Residual standard error: 55.1 on 186 degrees of freedom
## Multiple R-squared:  0.0777, Adjusted R-squared:  0.043
## F-statistic: 2.24 on 7 and 186 DF,  p-value: 0.0329
```

Notice: None of the **individual** p-values for the coefficients are significant.

Here is how we can access the **RSS** from each model.

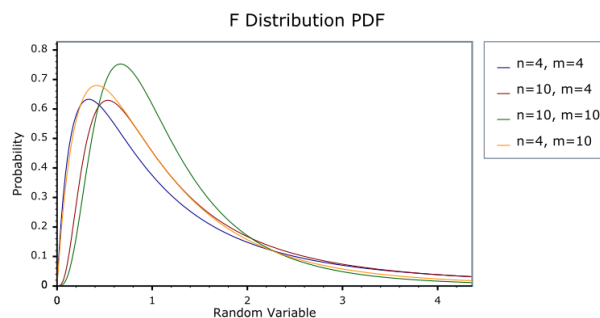
```
rss_full <- sum(resid(breed_model_both_vars)^2)
rss_restricted <- sum(resid(breed_model_other_dogs)^2)
```

So, our hypothesis is as follows:

$$H_0 : \beta \text{ of } good_with_young_children_level_i = 0, \text{ for all } i$$

Q9: Manually compute the incremental F-statistic for this null hypothesis. Then get the p-value for the corresponding F-distribution. Do we reject the null (at level $\alpha = 0.05$)?

Hint: `pf()` will give you the CDF of an F-statistic value. 1 minus the CDF will give the corresponding p-value. (ie, the probability that we observe a value **at least as extreme** as what we did, assuming the null hypothesis is true.) This is because the F-distribution is nonzero and has a single tail on the right:




```
## YOUR CODE HERE:
f_stat = ((rss_restricted - rss_full)/3)/(rss_full/(nrow(breed_data) - 8))

p_val = 1 - pf(f_stat,3,nrow(breed_data) - 8)

anova(breed_model_both_vars, breed_model_other_dogs)
```

```
## Analysis of Variance Table
##
## Model 1: rank_2020 ~ good_with_other_dogs + good_with_young_children
## Model 2: rank_2020 ~ good_with_other_dogs
##   Res.Df    RSS Df Sum of Sq   F Pr(>F)
## 1     186 565454
## 2     189 604447 -3     -38993 4.28 0.006 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
f_stat
```

```
## [1] 4.28
```

```
p_val
```

```
## [1] 0.00603
```

YOUR ANSWER HERE: We can reject the null hypothesis that the good with young children adds no information to our model.

You finished!